
OBJECT ORIENTED PROGRAMMING IN PYTHON

What is OOP?

Object-Oriented Programming (OOP) is a programming paradigm/approach/tareeka that organizes code into **objects** (instances of classes) that contain both **data (attributes)** and **behavior (methods)**. It helps make programs modular, reusable, and easier to maintain.

Solving a problem by creating an object

Difference Between OOP and Functional Programming

OOP	Functional Programming
Organizes code into objects and classes.	Organizes code into functions.
Data and methods are bundled together.	Data and functions are kept separate.
Uses concepts like inheritance and polymorphism.	Focuses on pure functions and immutability.
Objects can maintain and modify state.	Avoids changing state whenever possible.
Best for large, complex applications.	Best for data transformation and mathematical computations.

Why is it called DRY?

DRY stands for **Don't Repeat Yourself**.

It means **avoid writing the same code multiple times**. Instead, write reusable code using functions, classes, methods, or modules.

Example:

Instead of writing the same logic in many places, create one function or class and reuse it whenever needed. This reduces duplication, makes code easier to maintain, and minimizes errors.

What is a Class?

A **class** is a **blueprint or template** used to create objects. It defines the **attributes (data)** and **methods (functions)** that the objects will have.

Real-world analogy (School Form)

Imagine a **school admission form**.

The form is the **class** because it defines what information every student should have, such as:

- Name
- Roll Number
- Class
- Age

When **Ali** fills out the form, he becomes one **object**.

When **Maryam** fills out the same form, she becomes another **object**.

The form (class) is created **once**, but many students (objects) can be created from it.

Class	Object
School Admission Form	Ali's Filled Form
School Admission Form	Maryam's Filled Form

Easy way to remember:

- **Class = Blueprint / Template**
- **Object = Real thing created from the class**

A **class** is a **blueprint or template** used to create objects. It defines the **attributes (data)** and **methods (functions)** that the objects will have. A class itself does not store actual values; instead, it provides a structure from which multiple objects can be created. For example, a **school admission form** is like a class because it contains fields such as **Name, Roll Number, Age, and Class**. When **Ali** fills out the form, it becomes Ali's admission record (an object). When **Maryam** fills out the same form, it becomes Maryam's admission record (another object). The form (class) remains the same, but each student's filled form (object) contains different information. Thus, a class acts as a blueprint for creating multiple objects with the same structure but different data.

Khaali form class hai or bhara huwa form object hai

Object

An **object** is an **instance of a class**. It is the real entity created from a class and occupies memory in the computer. A class is only a blueprint or template, while an object is the actual thing created using that blueprint. An object contains its own **attributes (data)** and can perform **methods (functions)** defined in the class.

Object and instance are same thing. If someone say create instance of employee class, they r asking to create object of employee class

So An object (or instance) is a real entity created from a class.

Example:

Suppose **Student** is a class. When we create **Ali** and **Maryam** from the Student class, Ali and Maryam are **objects**. Both have the same structure (name, roll number, age), but their values are different.

Why is memory allocated only after object instantiation?

A class is just a **blueprint** and does not store actual data. Memory is allocated only when an object is created because the object stores the real values.

Example:

```
class Student:  
    name = ""
```

No memory is allocated for any student yet.

```
ali = Student()
```

Now memory is allocated for the object **ali**.

What does "Objects can invoke methods without revealing implementation details" mean?

It means we can use an object's methods **without knowing how they are implemented internally**. We only need to know how to use them. This concept is called **Abstraction**. Also, the object's data is protected and managed through its methods, which is called **Encapsulation**.

Example:

```
account.deposit(500)
```

We only call `deposit()`. We do not need to know how the balance is updated inside the method.

Break it into parts:

- `account` → the **object** (your bank account)
- `deposit` → the **method** (a function inside the class)
- `500` → the amount of money you want to add (500 rupees, dollars, etc.)

"Hey account, add 500 to your balance."

Suppose your balance is Rs. 1000.

You write:

```
account.deposit(500)
```

Now your balance becomes:

```
1000 + 500 = 1500
```

You don't write:

```
account.balance = account.balance + 500
```

Instead, you simply tell the account:

"Deposit 500."

That's **Abstraction**.

You only know **how to use it**.

You don't need to know **how it works internally**.

Then what is Encapsulation?

Suppose the bank lets everyone do this:

```
account.balance = 100000000
```

Anyone could change the balance to anything. 😊

That's not safe.

Instead, the bank says:

"No. You cannot change the balance directly."

Use our method:

```
account.deposit(500)
```

or

```
account.withdraw(200)
```

These methods safely update the balance.

So Encapsulation means:

Keep the data protected and allow controlled access through methods.

Abstraction

Abstraction is the process of hiding the implementation details and showing only the essential features to the user.

Encapsulation

Encapsulation is the process of bundling data and methods into a single unit (class) and protecting the data from direct access.

- **Abstraction:** Hide the implementation, show only what the user needs.
- **Encapsulation:** Protect data by controlling access through methods.

Modelling a Problem in OOP

When solving a real-world problem using OOP, we identify three main things:

1. Noun → Class

A **noun** is a person, place, animal, or thing. In OOP, nouns become **classes** because they represent the main objects we want to model.

Definition:

A **class** is a blueprint or template used to create objects.

Examples:

- Employee
- Student
- Car
- Book
- Bank Account

2. Adjective → Attributes

An **adjective** describes a **noun**. In OOP, adjectives become **attributes** because they describe the properties or characteristics of a class.

Attributes are the variables that store information about an object.

Examples:

For an Employee:

- name
- age
- salary

For a Student:

- name
- roll_number
- grade

3. Verb → Methods

A **verb** describes an action. In OOP, verbs become **methods** because methods define what an object can do.

Definition:

A method is a function defined inside a class that performs an action.

Examples:

For an Employee:

- `getSalary()`
- `incrementSalary()`

For a Student:

- `study()`
- `submitAssignment()`
- `displayResult()`

Easy School Example

Imagine a **Student** in a school.

- **Noun (Class):** Student
- **Adjectives (Attributes):** name, roll number, age, class
- **Verbs (Methods):** `study()`, `attendClass()`, `submitAssignment()`, `takeExam()`

So, if **Ali** and **Maryam** are students:

- **Class:** Student
- **Objects:** Ali, Maryam
- **Attributes:** name, roll number, age
- **Methods:** `study()`, `attendClass()`, `submitAssignment()`

This is how we model real-world problems in Object-Oriented Programming.

Class Attribute

A **class attribute** is a variable that directly belongs to the **class**. It is **shared by all objects** of that class.

An attribute that belongs to the class rather than a particular object.

Example:

```
class Employee:
    company = "Google"    # Class attribute

emp1 = Employee()
emp2 = Employee()

print(emp1.company)    # Google
print(emp2.company)    # Google
```

Both `emp1` and `emp2` share the same `company` value.

Object (Instance) Attribute

An **object attribute** (or **instance attribute**) is a variable that belongs to a **specific object**. Each object can have its **own different value**.

Example:

```
class Employee:
    company = "Google"    # Class attribute

emp1 = Employee()
emp2 = Employee()

emp1.name = "Maryam"    # instance attribute
emp1.age = 20
emp2.name = "MAYA"

print(emp1.name, emp1.company, emp1.age)
```

Here, `name` is an **object attribute** because each object has its own value.

Difference

Class Attribute	Object (Instance) Attribute
Belongs to the class.	Belongs to a specific object.
Shared by all objects.	Different for each object.
Defined inside the class.	Usually created for an object.

Easy trick to remember

- **Class Attribute** → **Same for everyone** (e.g., company = "Google")
- **Object Attribute** → **Different for everyone** (e.g., Ali's name, Maryam's name)

Note: Instance attributes take preference over class attributes during assignment & retrieval.

Self Parameter (Exam Definition)

The **self** parameter is a reference to the **current object (instance)** of a class. It is used to access the object's attributes and methods inside the class.

Simple Example

```
class Student:
    def introduce(self):
        print("Hello!")
```

```
ali = Student()
ali.introduce()
```

Here, `self` refers to the object `ali`.

Example with Attributes

```
class Student:
    def set_name(self, name):
        self.name = name
```

```
ali = Student()
ali.set_name("Ali")
```

```
print(ali.name)
```

Here:

- `self.name` means **the name attribute of the current object**.
- Since `self` refers to `ali`, it becomes:

```
ali.name = "Ali"
```

Easy way to remember

Think of `self` as meaning:

"This current object."

If you have:

```
ali = Student()
maryam = Student()
```

Then:

- When `ali` calls a method → `self = ali`
- When `maryam` calls a method → `self = maryam`

Python automatically passes `self`; you **don't** write it when calling the method.

One-line exam definition

`self` is a reference to the current object (instance) of a class and is used to access its attributes and methods.

@Realmayambano